

AI Assisted Coding Assignment- 13

RUDROJU RUPA SRI

2303A51918

BATCH-30

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

Identify code smells and inefficiencies in legacy Python scripts.

Use AI-assisted coding tools to refactor for readability, maintainability, and performance.

Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Code Duplication)

Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

Instructions:

Prompt AI to identify duplicate logic and replace it with functions or classes.

Ensure the refactored code maintains the same output.

Add docstrings to all functions.

Sample Legacy Code:

Legacy script with repeated logic

```
print("Area of Rectangle:", 5 * 10)
```

```
print("Perimeter of Rectangle:", 2 * (5 + 10))
```

```
print("Area of Rectangle:", 7 * 12)
```

```
print("Perimeter of Rectangle:", 2 * (7 + 12))
```

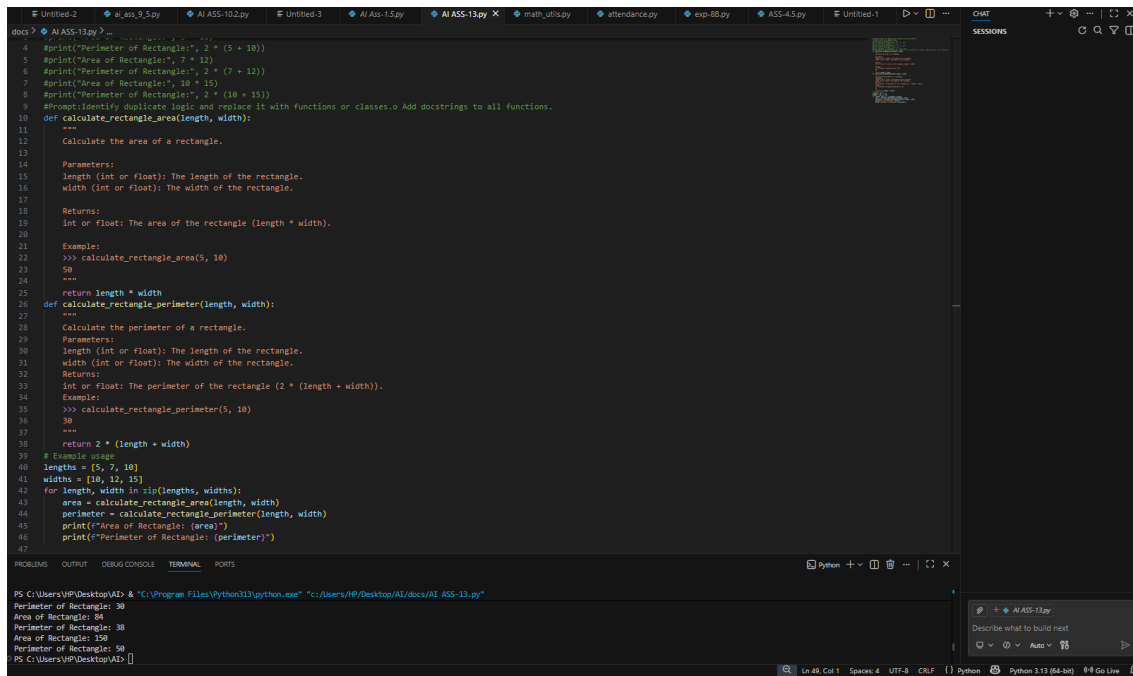
```
print("Area of Rectangle:", 10 * 15)
```

```
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Expected Output:

Refactored code with a reusable function and no duplication.

Well documented code



```
4 #print("Perimeter of Rectangle:", 2 * (5 + 10))
5 #print("Area of Rectangle:", 7 * 12)
6 #print("Perimeter of Rectangle:", 2 * (7 + 12))
7 #print("Area of Rectangle:", 10 * 15)
8 #print("Perimeter of Rectangle:", 2 * (10 + 15))
9 #Prompt: Identify duplicate logic and replace it with functions or classes. Add docstrings to all functions.
10 def calculate_rectangle_area(length, width):
11     """
12     Calculate the area of a rectangle.
13
14     Parameters:
15     length (int or float): The length of the rectangle.
16     width (int or float): The width of the rectangle.
17
18     Returns:
19     int or float: The area of the rectangle (length * width).
20
21     Example:
22     >>> calculate_rectangle_area(5, 10)
23     50
24     """
25     return length * width
26 def calculate_rectangle_perimeter(length, width):
27     """
28     Calculate the perimeter of a rectangle.
29
30     Parameters:
31     length (int or float): The length of the rectangle.
32     width (int or float): The width of the rectangle.
33
34     Returns:
35     int or float: The perimeter of the rectangle (2 * (length + width)).
36
37     Example:
38     >>> calculate_rectangle_perimeter(5, 10)
39     30
40     """
41     return 2 * (length + width)
42 # Example usage
43 lengths = [5, 7, 10]
44 widths = [10, 12, 15]
45 for length, width in zip(lengths, widths):
46     area = calculate_rectangle_area(length, width)
47     perimeter = calculate_rectangle_perimeter(length, width)
48     print(f"Area of Rectangle: {area}")
49     print(f"Perimeter of Rectangle: {perimeter}")
50
```

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python311\python.exe" "c:\Users\HP\Desktop\AI\docs\AI_Ass-13.py"
Perimeter of Rectangle: 30
Area of Rectangle: 54
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 58
PS C:\Users\HP\Desktop\AI>
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

Instructions:

Identify repeated or related logic and extract it into reusable functions.

Ensure the refactored code is modular, easy to read, and documented with docstrings.

Sample Legacy Code:

Legacy script with inline repeated logic

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Expected Output:

Code with a function `calculate_total(price)` that can be reused for multiple price inputs.

Well documented code

```
docs > AI ASS-13.py > ...
49 #Task Description #2 (Refactoring - Extracting Reusable Functions)
50 # Legacy script with inline repeated logic
51 #price = 250
52 #tax = price * 0.18
53 #total = price + tax
54 #print("Total Price:", total)
55 #price = 500
56 #tax = price * 0.18
57 #total = price + tax
58 #print("Total Price:", total)
59 #Prompt:Execute a Well documented Code with a function calculate_total(price) that can be reused for multiple price inputs.
60 def calculate_total(price):
61     """
62     Calculate the total price including tax.
63
64     Parameters:
65     price (int or float): The original price of the item.
66
67     Returns:
68     int or float: The total price after adding 18% tax.
69
70     Example:
71     >>> calculate_total(250)
72     295.0
73     """
74     tax = price * 0.18 # Calculate tax as 18% of the price
75     total = price + tax # Add tax to the original price to get the total
76     return total
77 # Example usage
78 prices = [250, 500]
79 for price in prices:
80     total_price = calculate_total(price)
81     print(f"Total Price: {total_price}")
```

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI ASS-13.py"
Total Price: 295.0
Total Price: 590.0
PS C:\Users\HP\Desktop\AI>
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

Class Name: GradeCalculator

Method Name: calculate_grade(self, marks)

The method must:

Accept marks as a parameter.

Return the corresponding grade as a string.

The grading logic must strictly follow the conditions below:

Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"

Marks $\geq 80 \rightarrow$ "Grade B"

Marks $\geq 70 \rightarrow$ "Grade C"

Marks $\geq 40 \rightarrow$ "Grade D"

Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

Include proper docstrings for:

The class

The method (with parameter and return descriptions)

The method must be reusable and called multiple times without rewriting conditional logic.

- Given code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Expected Output:

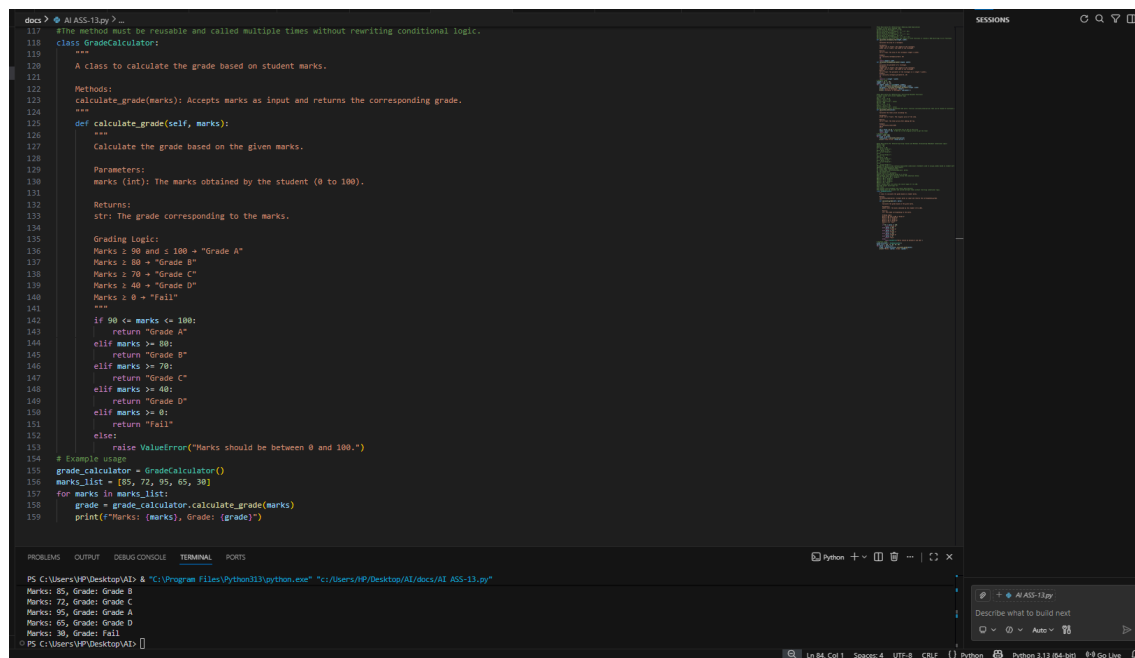
Define a class named GradeCalculator.

Implement a method calculate_grade(self, marks) inside the class.

Create an object of the class.

Call the method for different student marks.

Print the returned grade values.



```
docs > AI ASS-13.py > ...
117 #The method must be reusable and called multiple times without rewriting conditional logic.
118 class GradeCalculator:
119     """
120     A class to calculate the grade based on student marks.
121     """
122     Methods:
123     calculate_grade(marks): Accepts marks as input and returns the corresponding grade.
124     """
125     def calculate_grade(self, marks):
126         """
127         Calculate the grade based on the given marks.
128
129         Parameters:
130         marks (int): The marks obtained by the student (0 to 100).
131
132         Returns:
133         str: The grade corresponding to the marks.
134
135         Grading Logic:
136         Marks >= 90 and <= 100 -> "Grade A"
137         Marks >= 80 -> "Grade B"
138         Marks >= 70 -> "Grade C"
139         Marks >= 40 -> "Grade D"
140         Marks < 0 -> "Fail"
141         """
142         if 90 <= marks <= 100:
143             return "Grade A"
144         elif marks >= 80:
145             return "Grade B"
146         elif marks >= 70:
147             return "Grade C"
148         elif marks >= 40:
149             return "Grade D"
150         elif marks >= 0:
151             return "Fail"
152         else:
153             raise ValueError("Marks should be between 0 and 100.")
154
155 # Sample usage
156 grade_calculator = GradeCalculator()
157 marks_list = [85, 72, 95, 65, 30]
158 for marks in marks_list:
159     grade = grade_calculator.calculate_grade(marks)
160     print(f"Marks: {marks}, Grade: {grade}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\VP\Desktop\AD> & "C:\Program Files\Python313\python.exe" "C:\Users\VP\Desktop\AI\ASS-13.py"
Marks: 85, Grade: Grade B
Marks: 72, Grade: Grade C
Marks: 95, Grade: Grade A
Marks: 65, Grade: Grade D
Marks: 30, Grade: Fail
PS C:\Users\VP\Desktop\AD>
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input-processing logic into functions.

Instructions:

- o Identify input, processing, and output sections.
- o Convert each into a separate function.
- o Improve code readability without changing behavior.

- Sample Legacy Code:

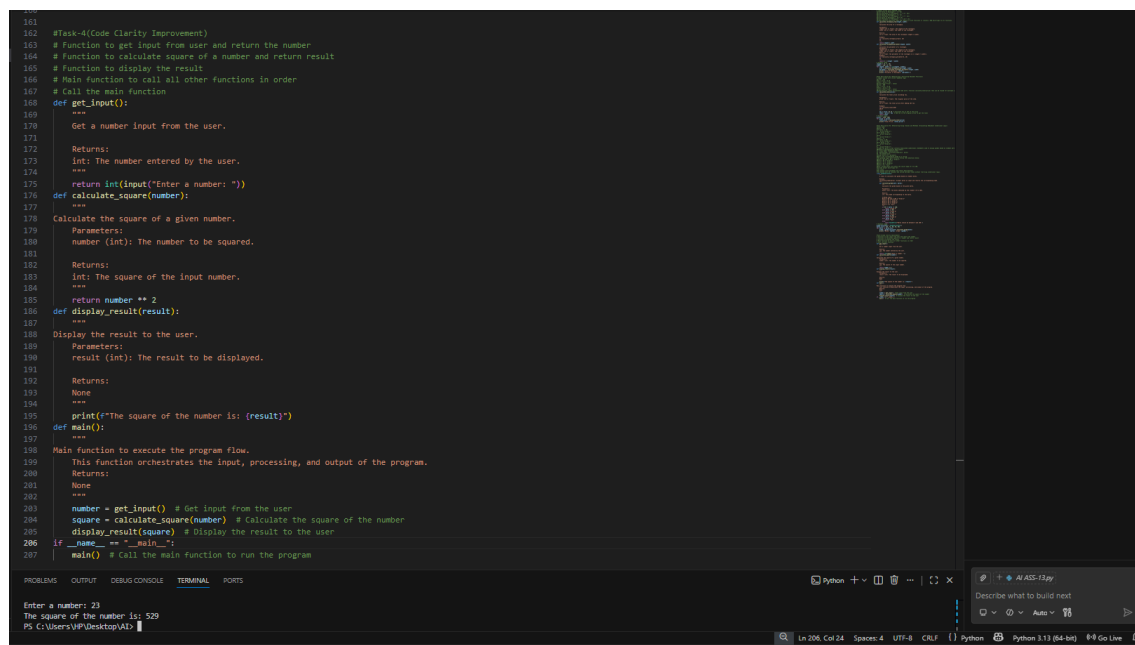
```
num = int(input("Enter number: "))
```

```
square = num * num
```

```
print("Square:", square)
```

- Expected Output:

- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.



```
161
162 #Task-4(Code Clarity Improvement)
163 # Function to get input from user and return the number
164 # Function to calculate square of a number and return result
165 # Function to display the result
166 # Main function to call all other functions in order
167 # Call the main function
168
169 def get_input():
170     """
171     Get a number input from the user.
172
173     Returns:
174     int: The number entered by the user.
175     """
176     return int(input("Enter a number: "))
177
178 def calculate_square(number):
179     """
180     Calculate the square of a given number.
181
182     Parameters:
183     number (int): The number to be squared.
184
185     Returns:
186     int: The square of the input number.
187     """
188     return number ** 2
189
190 def display_result(result):
191     """
192     Display the result to the user.
193
194     Parameters:
195     result (int): The result to be displayed.
196
197     Returns:
198     None
199     """
200     print(f"The square of the number is: {result}")
201
202 def main():
203     """
204     Main function to execute the program flow.
205     This function orchestrates the input, processing, and output of the program.
206
207     Returns:
208     None
209     """
210     number = get_input() # Get input from the user
211     square = calculate_square(number) # Calculate the square of the number
212     display_result(square) # Display the result to the user
213
214 if __name__ == "__main__":
215     main() # Call the main function to run the program
```

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

```
salary = 50000
```

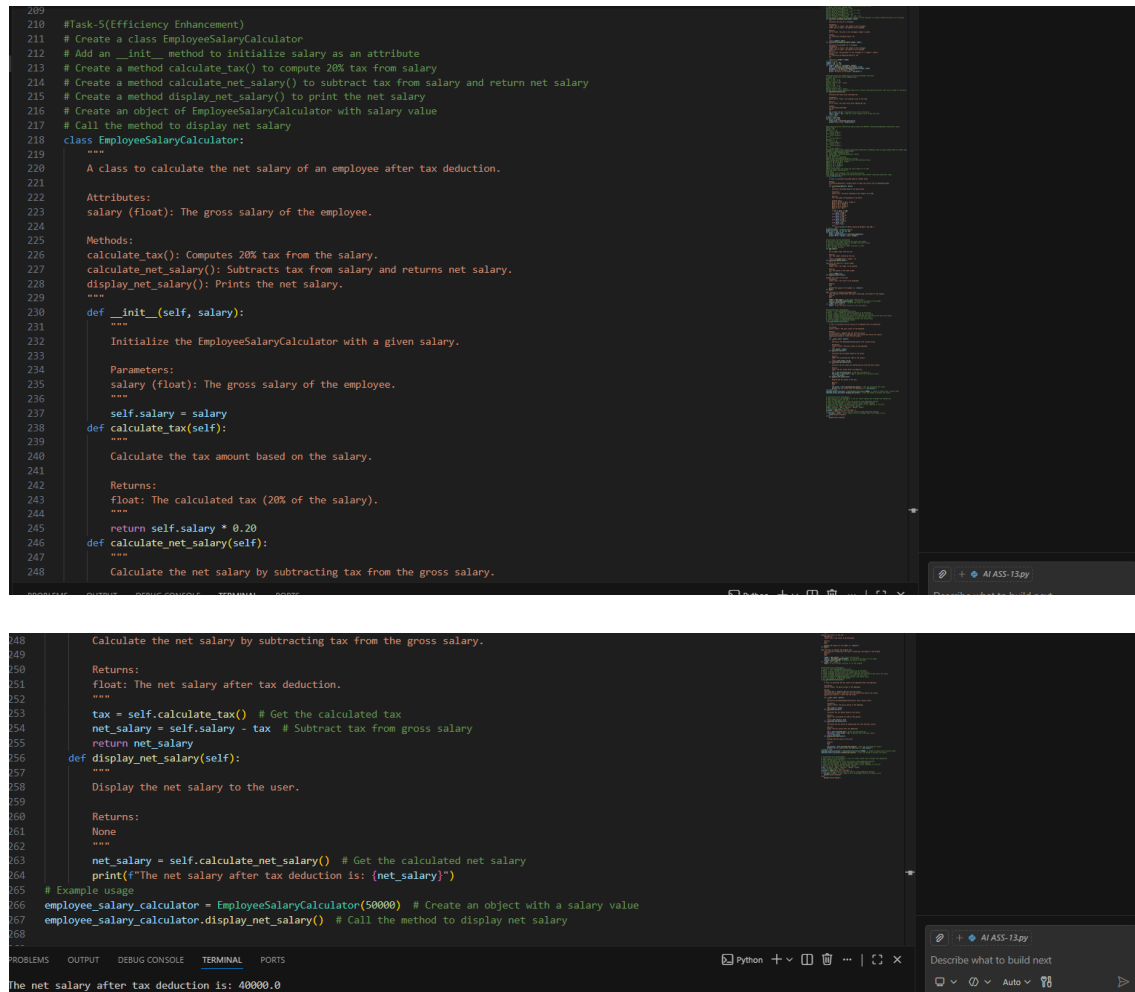
```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.



```
209
210 #Task-5(Efficiency Enhancement)
211 # Create a class EmployeeSalaryCalculator
212 # Add an __init__ method to initialize salary as an attribute
213 # Create a method calculate_tax() to compute 20% tax from salary
214 # Create a method calculate_net_salary() to subtract tax from salary and return net salary
215 # Create a method display_net_salary() to print the net salary
216 # Create an object of EmployeeSalaryCalculator with salary value
217 # Call the method to display net salary
218 class EmployeeSalaryCalculator:
219     """
220     A class to calculate the net salary of an employee after tax deduction.
221
222     Attributes:
223     salary (float): The gross salary of the employee.
224
225     Methods:
226     calculate_tax(): Computes 20% tax from the salary.
227     calculate_net_salary(): Subtracts tax from salary and returns net salary.
228     display_net_salary(): Prints the net salary.
229     """
230     def __init__(self, salary):
231         """
232         Initialize the EmployeeSalaryCalculator with a given salary.
233
234         Parameters:
235         salary (float): The gross salary of the employee.
236         """
237         self.salary = salary
238     def calculate_tax(self):
239         """
240         Calculate the tax amount based on the salary.
241
242         Returns:
243         float: The calculated tax (20% of the salary).
244         """
245         return self.salary * 0.20
246     def calculate_net_salary(self):
247         """
248         Calculate the net salary by subtracting tax from the gross salary.
249
250         Returns:
251         float: The net salary after tax deduction.
252         """
253         tax = self.calculate_tax() # Get the calculated tax
254         net_salary = self.salary - tax # Subtract tax from gross salary
255         return net_salary
256     def display_net_salary(self):
257         """
258         Display the net salary to the user.
259
260         Returns:
261         None
262         """
263         net_salary = self.calculate_net_salary() # Get the calculated net salary
264         print(f"The net salary after tax deduction is: {net_salary}")
265
266 # Example usage
267 employee_salary_calculator = EmployeeSalaryCalculator(50000) # Create an object with a salary value
268 employee_salary_calculator.display_net_salary() # Call the method to display net salary
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
```

```
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
```

The net salary after tax deduction is: 40000.0

Task 6 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity

- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

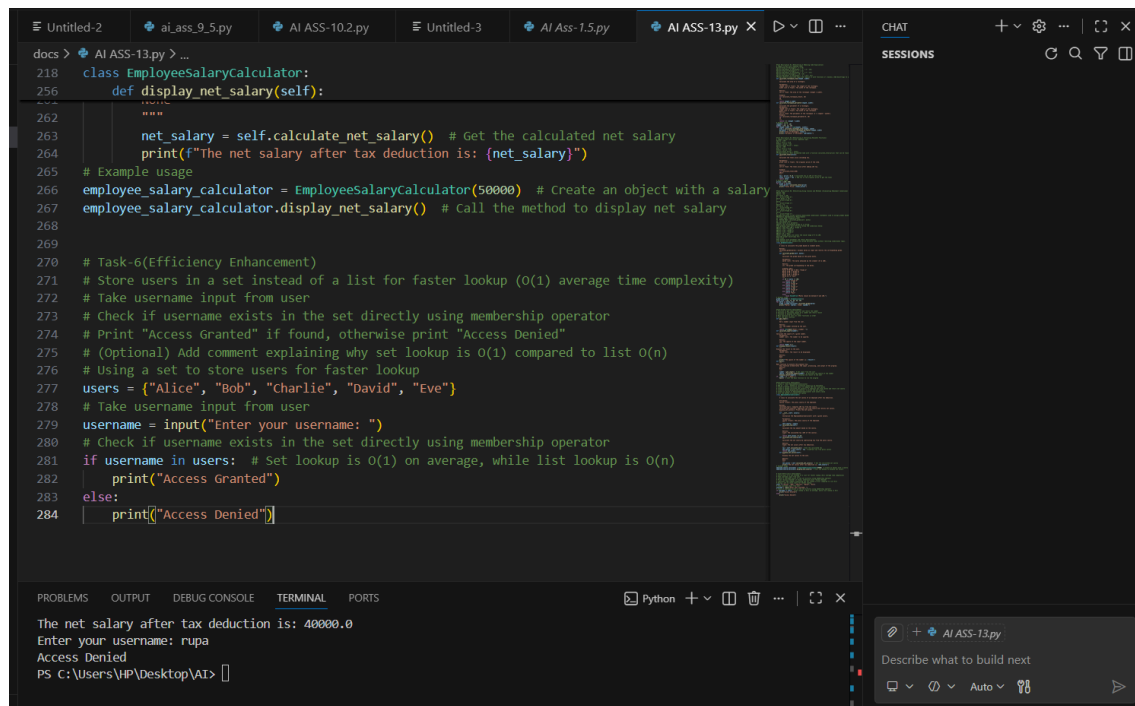
```
if u == name:
```

```
found = True
```

print("Access Granted" if found else "Access Denied")

Expected Outcome:

- o Use of sets or dictionaries with complexity justification



```
docs > AI ASS-13.py > ...
218 class EmployeeSalaryCalculator:
256     def display_net_salary(self):
262         net_salary = self.calculate_net_salary() # Get the calculated net salary
263         print(f"The net salary after tax deduction is: {net_salary}")
264     # Example usage
265 employee_salary_calculator = EmployeeSalaryCalculator(50000) # Create an object with a salary
266 employee_salary_calculator.display_net_salary() # Call the method to display net salary
267
268
269
270 # Task-6(Efficiency Enhancement)
271 # Store users in a set instead of a list for faster lookup (O(1) average time complexity)
272 # Take username input from user
273 # Check if username exists in the set directly using membership operator
274 # Print "Access Granted" if found, otherwise print "Access Denied"
275 # (Optional) Add comment explaining why set lookup is O(1) compared to list O(n)
276 # Using a set to store users for faster lookup
277 users = {"Alice", "Bob", "Charlie", "David", "Eve"}
278 # Take username input from user
279 username = input("Enter your username: ")
280 # Check if username exists in the set directly using membership operator
281 if username in users: # Set lookup is O(1) on average, while list lookup is O(n)
282     print("Access Granted")
283 else:
284     print("Access Denied")
```

The net salary after tax deduction is: 40000.0
Enter your username: rupa
Access Denied
PS C:\Users\HP\Desktop\AI>

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

Contains repeated conditional logic

Does not use reusable functions

Lacks documentation

Uses print-based procedural execution

Does not follow modular programming principles

Your task is to refactor the code into a proper format

Create a module `library.py` with functions:

`add_book(title, author, isbn)`

`remove_book(isbn)`

`search_book(isbn)`

Insert triple quotes under each function and let Copilot complete the docstrings.

Generate documentation in the terminal.

Export the documentation in HTML format.

Open the file in a browser.

Given Code

Library Management System (Unstructured Version)

This code needs refactoring into a proper module with documentation

```
library_db = {}

Adding first book

title = "Python Basics"
author = "John Doe"
isbn = "101"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Adding second book (duplicate logic)

title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)

isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)

isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")

# Searching again

isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
```



```
docs > library.py > add_book
1 """
2 Library Management Module
3 Provides functions to add, remove, and search books using ISBN.
4 """
5
6 # Dictionary to store books (ISBN as key)
7 library_db = {}
8
9
10 def add_book(title, author, isbn):
11     """
12     Adds a new book to the library.
13
14     Parameters:
15         title (str): Title of the book
16         author (str): Author of the book
17         isbn (str): Unique ISBN number
18
19     Returns:
20         str: Success or duplicate message
21     """
22     if isbn not in library_db:
23         library_db[isbn] = {"title": title, "author": author}
24         return "Book added successfully."
25     return "Book already exists."
26
27
28 def remove_book(isbn):
29     """
30     Removes a book from the library using ISBN.
31
32     Parameters:
33         isbn (str): ISBN of the book
34
35     Returns:
36         str: Success or not found message
37     """
38     if isbn in library_db:
39         del library_db[isbn]
40         return "Book removed successfully."
41     return "Book not found."
42
43
44 def search_book(isbn):
45     """
46     Searches for a book using ISBN.
47
48     Returns:
49         dict or str: Book details if found, otherwise not found message
50     """
51     if isbn in library_db:
52         return library_db[isbn]
53     return "Book not found."
```

```
docs > library.py > add_book
28 def remove_book(isbn):
29     """
30     Parameters:
31         isbn (str): ISBN of the book
32
33     Returns:
34         str: Success or not found message
35     """
36     if isbn in library_db:
37         del library_db[isbn]
38         return "Book removed successfully."
39     return "Book not found."
40
41
42 def search_book(isbn):
43     """
44     Searches for a book using ISBN.
45
46     Parameters:
47         isbn (str): ISBN of the book
48
49     Returns:
50         dict or str: Book details if found, otherwise not found message
51     """
52     if isbn in library_db:
53         return library_db[isbn]
54     return "Book not found."
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

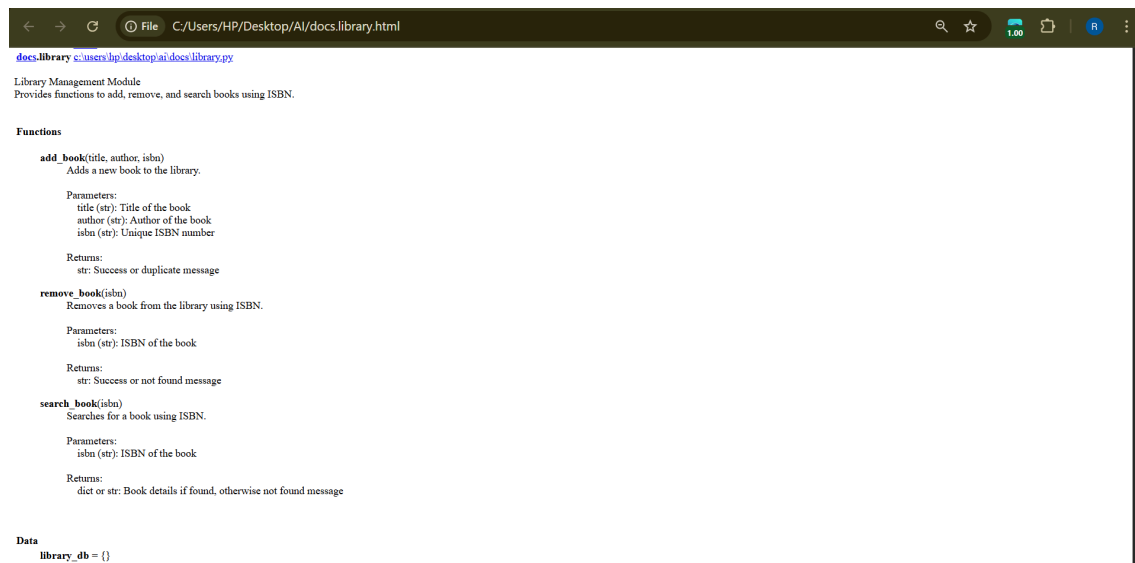
Marks: 65, Grade: Grade D
Marks: 30, Grade: Fail
Enter a number: 23
Enter a number: 23
The square of the number is: 529 -
Use help() to get the interactive help utility.
Use help(str) for help on the str class.

PS C:\Users\VP\Desktop\AD> py -m pydoc -w library

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\VP\Desktop\AD> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\VP\Desktop\AD> start docs.library.html
PS C:\Users\VP\Desktop\AD> start chrome docs.library.html
PS C:\Users\VP\Desktop\AD> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\VP\Desktop\AD> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\VP\Desktop\AD> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\VP\Desktop\AD> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\VP\Desktop\AD> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\VP\Desktop\AD> start chrome docs.library.html
PS C:\Users\VP\Desktop\AD> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\VP\Desktop\AD> python -m pydoc -w docs.library
wrote docs.library.html
PS C:\Users\VP\Desktop\AD> start chrome docs.library.html
```

Python 3.11 (64-bit) 98 Go Live



Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

Global variables.

Inefficient loop.

No functions or modularity.

Task for Students:

Refactor into a clean reusable function (generate_fibonacci).

Add docstrings and test cases.

Compare AI-refactored vs original.

Bad Code Version:

#fibonacci bad version

```
n=int(input("Enter limit: "))
```

```
a=0
```

```
b=1
```

```
print(a)
```

```
print(b)
```

```
for i in range(2,n):
```

```
    c=a+b
```

```
    print(c)
```

```
    a=b
```

```
    b=c
```

```
docs > AI-ASS-13.py > ...
290 # If n == 1 return [0]
291 # Initialize list with first two values 0 and 1
292 # Use loop to generate remaining Fibonacci numbers
293 # Append new value as sum of last two numbers
294 # Return the Fibonacci list
295 # Take input from user
296 # Call the function and print the result
297 # Add few test cases to check different values of n
298 def generate_fibonacci(n):
299     """
300     Generate a list of Fibonacci numbers up to the nth number.
301
302     Parameters:
303     n (int): The number of Fibonacci numbers to generate.
304
305     Returns:
306     list: A list containing the Fibonacci sequence up to n.
307
308     Example:
309     >>> generate_fibonacci(10)
310     [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
311     """
312     if n <= 0:
313         return []
314     elif n == 1:
315         return [0]
316
317     fib_sequence = [0, 1] # Initialize list with first two values
318     for i in range(2, n):
319         next_value = fib_sequence[-1] + fib_sequence[-2] # Sum of last two numbers
320         fib_sequence.append(next_value) # Append new value to the list
321
322     return fib_sequence
323
324 # Take input from user
325 n = int(input("Enter the number of Fibonacci numbers to generate: "))
326 # Call the function and print the result
327 fibonacci_numbers = generate_fibonacci(n)
328 print(f"Fibonacci sequence up to (n) numbers: {fibonacci_numbers}")
329 # Add few test cases to check different values of n
330 print(generate_fibonacci(0)) # Should return []
331 print(generate_fibonacci(1)) # Should return [0]
332 print(generate_fibonacci(5)) # Should return [0, 1, 1, 2, 3]
333 print(generate_fibonacci(10)) # Should return [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
334
335
PS C:\Users\VP\Desktop\AI> "C:\Program Files\Python311\python.exe" "C:\Users\VP\Desktop\AI\docs\AI-ASS-13.py"
Enter the number of Fibonacci numbers to generate: 23
Fibonacci sequence up to 23 numbers: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17713]
[1]
[0]
[0, 1, 1, 2, 3]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\VP\Desktop\AI>
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

Inefficient prime checking.

No functions.

Hardcoded inputs.

Task for Students:

Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.

Add docstrings and optimize.

Generate a list of twin primes in a given range using AI.

Bad Code Version:

twin primes bad version

```
a=11
```

```
b=13
```

```
fa=0
```

```
for i in range(2,a):
```

```
if a%i==0:
```

```
fa=1
```

```
fb=0
```

```
for i in range(2,b):
```

```
if b%i==0:
```

```
fb=1
```

```
if fa==0 and fb==0 and abs(a-b)==2:
```

```

print("Twin Primes")
else:
print("Not Twin Primes")

```

```

335 #Task-9(Refactoring - Modularizing Code with Functions)
336 # Create a function is_prime(n)
337 # Add triple quotes for docstring inside is_prime
338 # If n <= 1 return False
339 # Check divisibility from 2 to square root of n for optimization
340 # Return True if no divisors found else False
341 # Create a function is_twin_prime(p1, p2)
342 # Add triple quotes for docstring inside is_twin_prime
343 # Check if both numbers are prime using is_prime
344 # Check if absolute difference between them is 2
345 # Return True if twin primes else False
346 # Take input range from user (start and end)
347 # Generate list of twin prime pairs in given range
348 # Print the twin prime pairs
349
350 def is_prime(n):
351     """
352     Check if a number is prime.
353
354     Parameters:
355     n (int): The number to check for primality.
356
357     Returns:
358     bool: True if n is prime, False otherwise.
359
360     Example:
361     >>> is_prime(7)
362     True
363     >>> is_prime(10)
364     False
365     """
366     if n <= 1:
367         return False
368     for i in range(2, int(n**0.5) + 1): # Check divisibility up to the square root of n
369         if n % i == 0:
370             return False
371     return True

```

```

370
371 def is_twin_prime(p1, p2):
372     """
373     Check if two numbers are twin primes.
374
375     Parameters:
376     p1 (int): The first number.
377     p2 (int): The second number.
378
379     Returns:
380     bool: True if p1 and p2 are twin primes, False otherwise.
381
382     Example:
383     >>> is_twin_prime(11, 13)
384     True
385     >>> is_twin_prime(17, 19)
386     True
387     >>> is_twin_prime(10, 12)
388     False
389     """
390     return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
391
392 # Take input range from user (start and end)
393 start = int(input("Enter the start of the range: "))
394 end = int(input("Enter the end of the range: "))
395 # Generate list of twin prime pairs in given range
396 twin_primes = []
397 for num in range(start, end + 1):
398     if is_prime(num) and is_prime(num + 2):
399         twin_primes.append((num, num + 2))
400 # Print the twin prime pairs
401 print(f"Twin prime pairs in the range {start} to {end}: {twin_primes}")

```

Terminal Output:
 PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "C:\Users\HP\Desktop\AI\ASS-13.py"
 Enter the start of the range: 1
 Enter the end of the range: 10
 Twin prime pairs in the range 1 to 10: [(3, 5), (5, 7)]
 PS C:\Users\HP\Desktop\AI>

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation. The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not

follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

Rat

Ox

Tiger

Rabbit

Dragon

Snake

Horse

Goat (Sheep)

Monkey

Rooster

Dog

Pig

Chinese Zodiac Program(Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))
```

```
if year % 12 == 0:
```

```
    print("Monkey")
```

```
elif year % 12 == 1:
```

```
    print("Rooster")
```

```
elif year % 12 == 2:
```

```
    print("Dog")
```

```
elif year % 12 == 3:
```

```
    print("Pig")
```

```
elif year % 12 == 4:
```

```
    print("Rat")
```

```
elif year % 12 == 5:
```

```
    print("Ox")
```

```
elif year % 12 == 6:
```

```
    print("Tiger")
```

```
elif year % 12 == 7:
```

```
    print("Rabbit")
```

```
elif year % 12 == 8:
```

```
    print("Dragon")
```

```
elif year % 12 == 9:
```

```
    print("Snake")
```

```
elif year % 12 == 10:
```

```
    print("Horse")
```

```
elif year % 12 == 11:
```

```
    print("Goat")
```

You must:

Create a reusable function: `get_zodiac(year)`
 Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
 Add proper docstrings.
 Separate input handling from logic.
 Improve readability and maintainability.
 Ensure output remains correct.

```

403 #Task-10
404 """
405 Chinese Zodiac Module
406 Refactored version using clean modular design.
407 """
408 def get_zodiac(year):
409     """
410     Returns the Chinese Zodiac sign for a given year.
411     Parameters:
412     year (int): The year to evaluate
413     Returns:
414     str: Corresponding Chinese Zodiac sign
415     """
416     zodiac_cycle = [
417         "Monkey", "Rooster", "Dog", "Pig",
418         "Rat", "Ox", "Tiger", "Rabbit",
419         "Dragon", "Snake", "Horse", "Goat"
420     ]
421     return zodiac_cycle[year % 12]
422
423 def main():
424     """
425     Handles user input and displays zodiac result.
426     """
427     year = int(input("Enter a year: "))
428     sign = get_zodiac(year)
429     print(sign)
430
431 if __name__ == "__main__":
432     main()
  
```

Terminal output:

```

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "C:\Users\HP\Desktop\AI\do
Enter a year: 2026
Horse
PS C:\Users\HP\Desktop\AI>
  
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

18 → 1 + 8 = 9 → 18 ÷ 9 = 2 ✓ (Harshad Number)

19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer ✗ (Not Harshad)

Problem Statement

The current implementation:

Mixes logic and input handling

Uses redundant variables

Does not use reusable functions properly

Returns print statements instead of boolean values

Lacks documentation

You must refactor the code to follow clean coding principles.

Harshad Number Checker(Unstructured Version)

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum_digits = sum_digits + digit
```

```
    temp = temp // 10
```

```
if sum_digits != 0:
```

```
    if num % sum_digits == 0:
```

```
        print("True")
```

```
    else:
```

```
        print("False")
```

```
    else:
```

```
        print("False")
```

You must:

Create a reusable function: `is_harshad(number)`

The function must:

Accept an integer parameter.

Return True if the number is divisible by the sum of its digits.

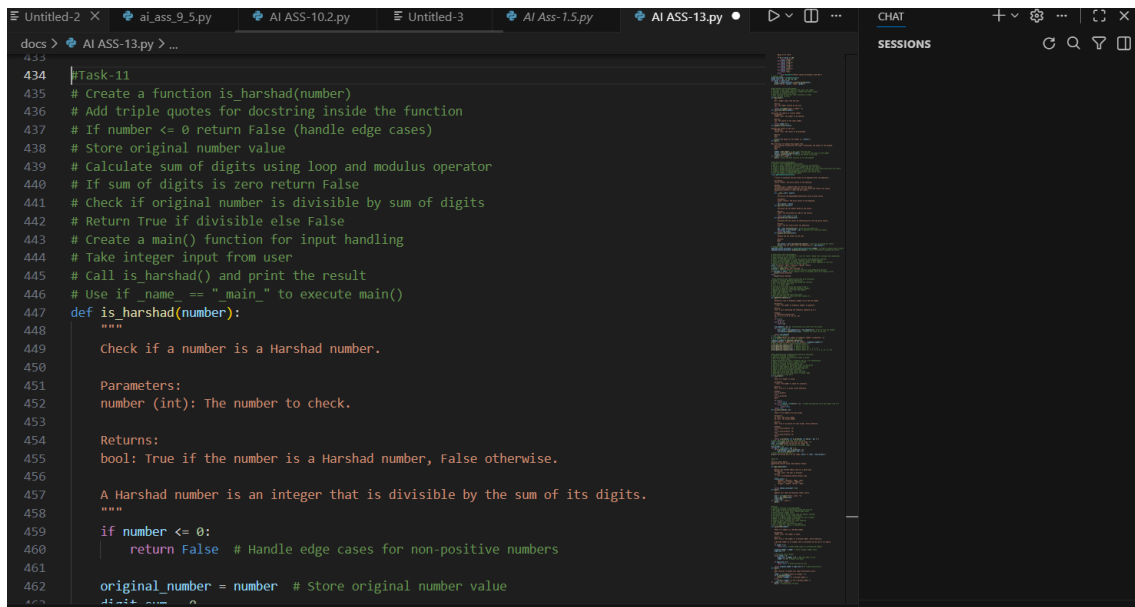
Return False otherwise.

Separate user input from core logic.

Add proper docstrings.

Improve readability and maintainability.

Ensure the program handles edge cases (e.g., 0, negative numbers).



```
434 Task-11
435 # Create a function is_harshad(number)
436 # Add triple quotes for docstring inside the function
437 # If number <= 0 return False (handle edge cases)
438 # Store original number value
439 # Calculate sum of digits using loop and modulus operator
440 # If sum of digits is zero return False
441 # Check if original number is divisible by sum of digits
442 # Return True if divisible else False
443 # Create a main() function for input handling
444 # Take integer input from user
445 # Call is_harshad() and print the result
446 # Use if __name__ == "__main__" to execute main()
447 def is_harshad(number):
448     """
449     Check if a number is a Harshad number.
450
451     Parameters:
452     number (int): The number to check.
453
454     Returns:
455     bool: True if the number is a Harshad number, False otherwise.
456
457     A Harshad number is an integer that is divisible by the sum of its digits.
458     """
459     if number <= 0:
460         return False # Handle edge cases for non-positive numbers
461
462     original_number = number # Store original number value
463     temp = number
464     sum_digits = 0
465     while temp > 0:
466         digit = temp % 10
467         sum_digits = sum_digits + digit
468         temp = temp // 10
469     if sum_digits != 0:
470         if number % sum_digits == 0:
471             return True
472         else:
473             return False
474     else:
475         return False
476
477 # Main function for input handling
478 def main():
479     # Take integer input from user
480     number = int(input("Enter a number: "))
481     # Call is_harshad() and print the result
482     result = is_harshad(number)
483     print(result)
484
485 # Use if __name__ == "__main__" to execute main()
486 if __name__ == "__main__":
487     main()
```

```
docs > AI ASS-13.py > ...
447 def is_harshad(number):
448
449     original_number = number # Store original number value
450     digit_sum = 0
451
452     # Calculate sum of digits
453     while number > 0:
454         digit_sum += number % 10 # Add last digit to sum
455         number //= 10 # Remove last digit
456
457     if digit_sum == 0:
458         return False # Avoid division by zero
459
460     return original_number % digit_sum == 0 # check divisibility
461
462 def main():
463     """
464     Main function to handle user input and display result.
465     """
466     number = int(input("Enter an integer: "))
467     if is_harshad(number):
468         print(f"{number} is a Harshad number.")
469     else:
470         print(f"{number} is not a Harshad number.")
471
472 if __name__ == "__main__":
473     main() # Execute main function
474
475
476
477
478
479
480
481
482
483
484
485
```

Enter an integer: 24
24 is a Harshad number.
PS C:\Users\VP\Desktop\AI>

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation. The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)

- Mixes input handling with business logic

- Uses print statements instead of return values

- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros(Unstructured Version)

```
n = int(input("Enter a number: "))
```

```
fact = 1
```

```
i = 1
```

```
while i <= n:
```

```
fact = fact * i
```

```
i = i + 1
```

```
count = 0
```

```
while fact % 10 == 0:
```

```
count = count + 1
```

```
fact = fact // 10
```

```
print("Trailing zeros:", count)
```


You must:

Create a reusable function: `count_trailing_zeros(n)`

The function must:

Accept a non-negative integer `n`.

Return the number of trailing zeros in `n!`.

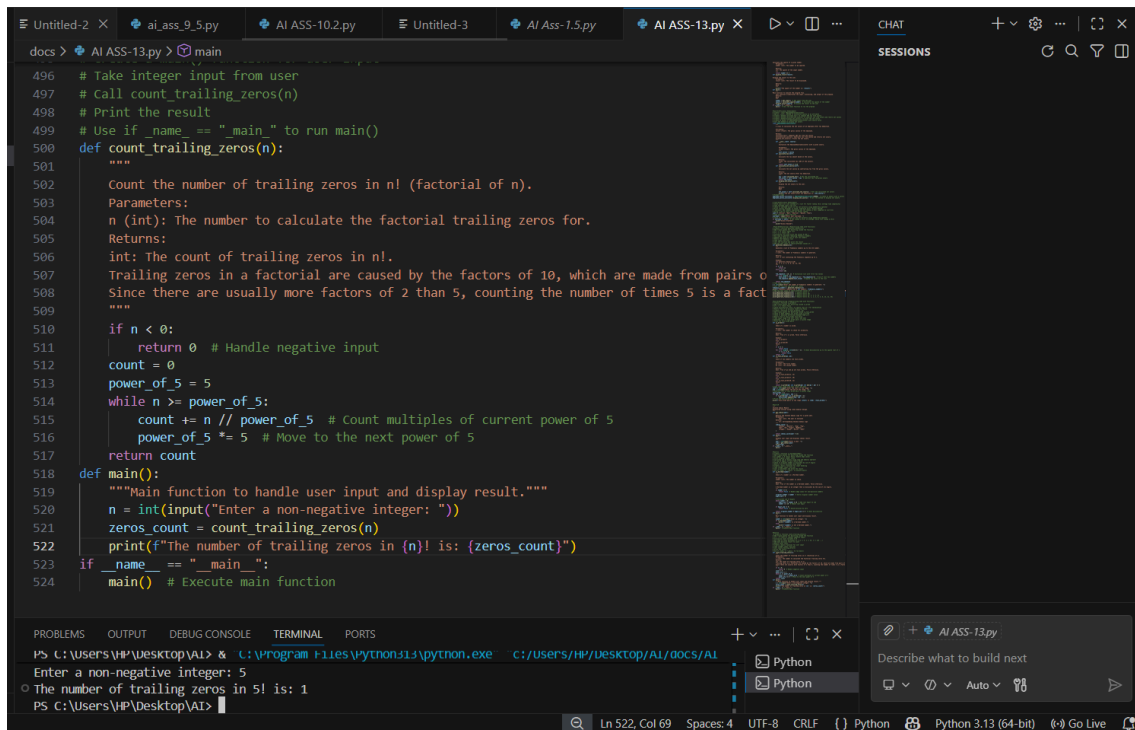
Do NOT compute the full factorial.

Use an optimized mathematical approach (count multiples of 5).

Add proper docstrings.

Separate user interaction from core logic.

Handle edge cases (e.g., negative numbers, zero).



```
docs > AI ASS-13.py > main
496 # Take integer input from user
497 # Call count_trailing_zeros(n)
498 # Print the result
499 # Use if __name__ == "__main__" to run main()
500 def count_trailing_zeros(n):
501     """
502     Count the number of trailing zeros in n! (factorial of n).
503     Parameters:
504     n (int): The number to calculate the factorial trailing zeros for.
505     Returns:
506     int: The count of trailing zeros in n!.
507     Trailing zeros in a factorial are caused by the factors of 10, which are made from pairs of 2 and 5.
508     Since there are usually more factors of 2 than 5, counting the number of times 5 is a factor is sufficient.
509     """
510     if n < 0:
511         return 0 # Handle negative input
512     count = 0
513     power_of_5 = 5
514     while n >= power_of_5:
515         count += n // power_of_5 # Count multiples of current power of 5
516         power_of_5 *= 5 # Move to the next power of 5
517     return count
518 def main():
519     """Main function to handle user input and display result."""
520     n = int(input("Enter a non-negative integer: "))
521     zeros_count = count_trailing_zeros(n)
522     print(f"The number of trailing zeros in {n}! is: {zeros_count}")
523 if __name__ == "__main__":
524     main() # Execute main function

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\Desktop\AI> & C:\Program Files\Python311\python.exe c:\users\hp/desktop/ai/docs/ai
Enter a non-negative integer: 5
The number of trailing zeros in 5! is: 1
PS C:\Users\HP\Desktop\AI>
```

Test Cases Design

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.

- Test Cases to Design:

Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$

Edge: $1 \rightarrow [1]$

Negative: -5

Large: 27 (well-known long sequence)

- Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

Takes an integer n as input.

Generates the Collatz sequence (also called the $3n+1$ sequence).

The rules are:

If n is even $\rightarrow$ next = $n / 2$.

If n is odd $\rightarrow$ next = $3n + 1$.

Repeat until we reach 1.

Return the full sequence as a list.

Example

Input: 6

Steps:

6 (even $\rightarrow 6/2 = 3$)

3 (odd $\rightarrow 3*3+1 = 10$)

10 (even $\rightarrow 10/2 = 5$)

5 (odd $\rightarrow 3*5+1 = 16$)

16 (even $\rightarrow 16/2 = 8$)

8 (even $\rightarrow 8/2 = 4$)

4 (even $\rightarrow 4/2 = 2$)

2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

```
def generate_collatz(n):
    """Generate the Collatz sequence starting from n.

    Parameters:
    n (int): The starting number for the Collatz sequence. Must be a positive integer.

    Returns:
    list: A list containing the Collatz sequence starting from n and ending at 1.

    The Collatz sequence is defined as follows:
    - If n is even, the next number is n / 2.
    - If n is odd, the next number is 3 * n + 1.
    The sequence continues until it reaches 1.
    """
    if n <= 0:
        raise ValueError("Input must be a positive integer.")
    sequence = [n] # Initialize list with starting value
    while n != 1:
        if n % 2 == 0: # If n is even
            n //= 2
        else: # If n is odd
            n = 3 * n + 1
        sequence.append(n) # Append each new value to the list
    return sequence

def main():
    """Main function to handle user input and display the Collatz sequence."""
    n = int(input("Enter a positive integer: "))
    try:
        collatz_sequence = generate_collatz(n)
        print(f"Collatz sequence starting from {n}: {collatz_sequence}")
    except ValueError as e:
        print(e)

if __name__ == "__main__":
    main() # Execute main function
```

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python311\python.exe" "C:/Users/HP/Desktop/AI/docs/AI_ASS-13.py"
Enter a positive integer: 8
Collatz sequence starting from 8: [8, 4, 2, 1]
PS C:\Users\HP\Desktop\AI>
```

Task 14 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:

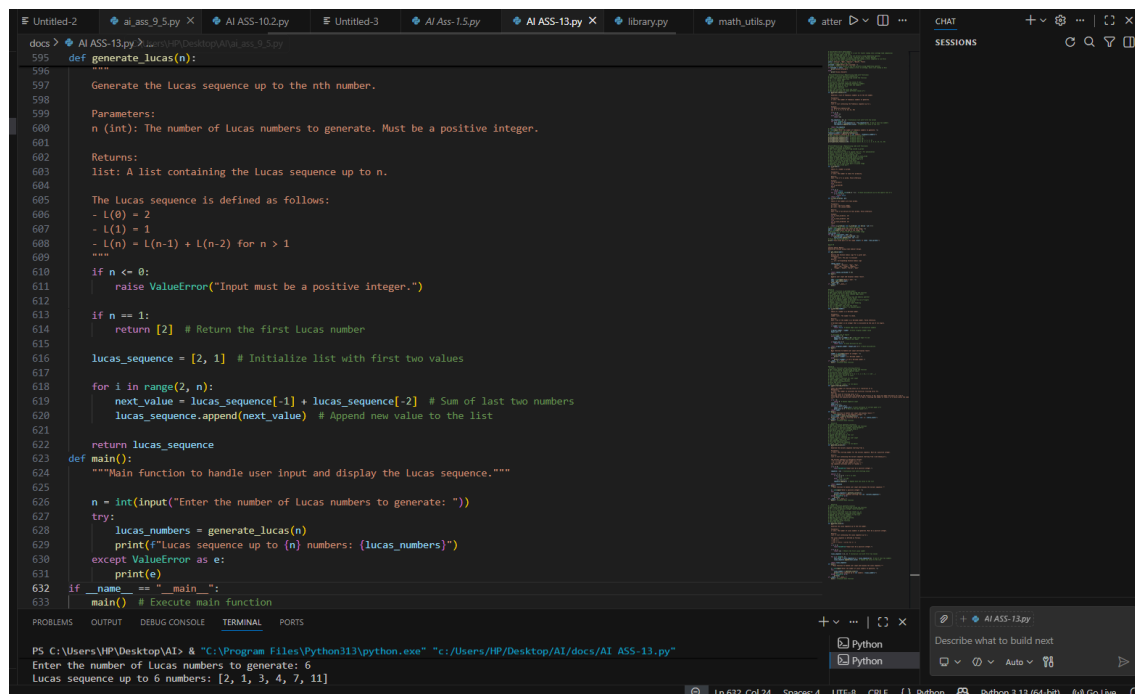
Normal: 5 → [2, 1, 3, 4, 7]

Edge: 1 → [2]

Negative: -5 → Error

Large: 10 (last element = 76).

- Requirement: Validate correctness with pytest.



```
docs > AI ASS-13.py > ...
395 def generate_lucas(n):
396     """
397     Generate the Lucas sequence up to the nth number.
398
399     Parameters:
400     n (int): The number of Lucas numbers to generate. Must be a positive integer.
401
402     Returns:
403     list: A list containing the Lucas sequence up to n.
404
405     The Lucas sequence is defined as follows:
406     - L(0) = 2
407     - L(1) = 1
408     - L(n) = L(n-1) + L(n-2) for n > 1
409     """
410     if n <= 0:
411         raise ValueError("Input must be a positive integer.")
412
413     if n == 1:
414         return [2] # Return the first Lucas number
415
416     lucas_sequence = [2, 1] # Initialize list with first two values
417
418     for i in range(2, n):
419         next_value = lucas_sequence[-1] + lucas_sequence[-2] # Sum of last two numbers
420         lucas_sequence.append(next_value) # Append new value to the list
421
422     return lucas_sequence
423
424 def main():
425     """Main function to handle user input and display the Lucas sequence."""
426
427     n = int(input("Enter the number of Lucas numbers to generate: "))
428     try:
429         lucas_numbers = generate_lucas(n)
430         print(f"Lucas sequence up to {n} numbers: {lucas_numbers}")
431     except ValueError as e:
432         print(e)
433
434 if __name__ == "__main__":
435     main() # Execute main function
```

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI ASS-13.py"

Enter the number of Lucas numbers to generate: 6

Lucas sequence up to 6 numbers: [2, 1, 3, 4, 7, 11]

Task 15 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.

- Test Cases to Design:

Normal: "hello" → (2,3)

Edge: "" → (0,0)

Only vowels: "aeiou" → (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

```
docs > AI ASS-13.py > ...
645 # Add main block for manual testing
646 # Take string input from user
647 # Call the function and print the result
648 def count_vowels_consonants(text):
649     """
650     Count the number of vowels and consonants in a given text.
651
652     Parameters:
653     text (str): The input string to analyze.
654
655     Returns:
656     tuple: A tuple containing the count of vowels and consonants (vowel_count, consonant_count).
657
658     This function counts the number of vowels (a, e, i, o, u) and consonants in the input text.
659     It ignores digits, spaces, and special characters.
660     """
661     vowel_count = 0
662     consonant_count = 0
663     vowels = "aeiou"
664
665     for char in text.lower(): # Convert text to lowercase for case-insensitive checking
666         if char in vowels:
667             vowel_count += 1 # Increase vowel count
668         elif char.isalpha(): # Check if character is an alphabet letter
669             consonant_count += 1 # Increase consonant count
670
671     return vowel_count, consonant_count # Return counts as a tuple
672 def main():
673     """Main function to handle user input and display vowel and consonant counts."""
674
675     text = input("Enter a string: ")
676     vowels, consonants = count_vowels_consonants(text)
677     print(f"Number of vowels: {vowels}")
678     print(f"Number of consonants: {consonants}")
679 if __name__ == "__main__":
680     main() # Execute main function

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI ASS-13.py"
Enter a string: rupasri
Number of vowels: 3
Number of consonants: 4
```